

MLPy Workshop 6

March 2, 2026

1 Week 6 - Logistic Regression

Student Name 1, Student Name 2

1.0.1 Aims

The main concepts covered in this notebook are:

- logistic regression
- performance metrics for classification
- dealing with imbalanced data
- multi-class logistic regression

1. Setup
2. Binary Logistic Regression
3. Regularization
4. Imbalanced Data
5. Multi-class Example

This week we will be implementing logistic regression for a classification task.

- We will mainly focus on the data set stored in `Default.csv`.
- For the multi-class example, we consider the `iris data` at the end of the notebook.

During workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. You should aim to switch roles between driver and navigator approximately every 15 minutes. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.
- In some Exercises, you will see some beneficial hints at the bottom of questions.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Setup

2.1 Packages

Let's load the packages we need for this workshop.

```
[1]: # Data libraries
import pandas as pd
import numpy as np

# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules
import sklearn
from sklearn.linear_model import LogisticRegression

# Other necessary packages
from sklearn.preprocessing import StandardScaler # scaling features
from sklearn.pipeline import make_pipeline      # combining classifier
↳ steps
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

from sklearn.model_selection import GridSearchCV, KFold, StratifiedKFold
from sklearn.model_selection import train_test_split
```

2.2 Helper Functions

We will make use of one of the helper functions that we saw last week.

```
[2]: def get_coefs(m, plot = False, feature_names = None, figsize = (5,5), figtitle =
↳ None, intercept = True):
    """Returns model coefficients in a data frame for a fitted linear model.

    Args:
        m: sklearn linear model object or pipeline with linear model as final
↳ step
        plot: boolean value, should coefficients be plotted with error bars
        feature_names: list of feature names to use in the plot
        figsize: tuple defining figure size
        figtitle: string defining figure title
        intercept: boolean value, should intercept be included in the plot
    """

    # Extract intercept and coefficients into a single array
```

```

w0 = m[-1].intercept_ if isinstance(m, sklearn.pipeline.Pipeline) else m.
↪intercept_
w1 = m[-1].coef_ if isinstance(m, sklearn.pipeline.Pipeline) else m.coef_
w = np.concatenate((w0.reshape(-1), w1.reshape(-1)))
# Extract name of features
if feature_names is None:
    feature_names = m[:-1].get_feature_names_out() if isinstance(m, sklearn.
↪pipeline.Pipeline) else m.feature_names_in_
    feature_names = np.concatenate(['intercept', feature_names])
# Create a data frame
w_df = pd.DataFrame({'feature': feature_names, 'coef': w}).sort_values_
↪("coef", ascending=False)

if plot:
    if not intercept:
        w_df = w_df[w_df['feature'] != 'intercept']
    plt.figure(figsize=figsize)
    plt.barh(w_df['feature'], w_df['coef'])
    plt.ylabel('Features')
    plt.xlabel('Coefficient Value')
    plt.axvline(x=0, color=".5")
    if figtitle is not None:
        plt.title(figtitle)
    plt.grid()
    plt.show()

return w_df

```

2.3 Data

The dataset collects information on **10000** individuals, recording whether they defaulted on their credit card or not as well as other characteristics. Specifically, the included columns in the data are:

- **default** - Whether the individual has defaulted
- **student** - Whether the individual is the student
- **balance** - The balance in the individual's account
- **income** - Income of an individual

Our aim is to build a model using Logistic Regression to predict if person will default or not.

```

[3]: df_default = pd.read_csv("Default.csv", index_col=0)

df_default.head()

```

```

[3]:  default student      balance      income
1      No        No    729.526495  44361.62507

```

2	No	Yes	817.180407	12106.13470
3	No	No	1073.549164	31767.13895
4	No	No	529.250605	35704.49394
5	No	No	785.655883	38463.49588

2.4 Exploratory Data Analysis

We will start with explanatory data analysis (EDA) to get more insight about the data.

2.4.1 Exercise 1 (CORE)

Examine the structure of the data. Consider the following questions:

- What are the types of each variable? Based on the descriptive statistics, do anticipate any feature engineering steps that may be needed? Are there any missing values in the data?
- Visualize the features (balance, income, student) and comment on any differences that you observe between individuals that have defaulted

```
[4]: # Info on data types
df_default.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 10000 entries, 1 to 10000
Data columns (total 4 columns):
#   Column    Non-Null Count  Dtype
---  -
0   default    10000 non-null    object
1   student    10000 non-null    object
2   balance    10000 non-null    float64
3   income     10000 non-null    float64
dtypes: float64(2), object(2)
memory usage: 390.6+ KB
```

```
[5]: # Descriptive statistics
print(df_default.describe())
```

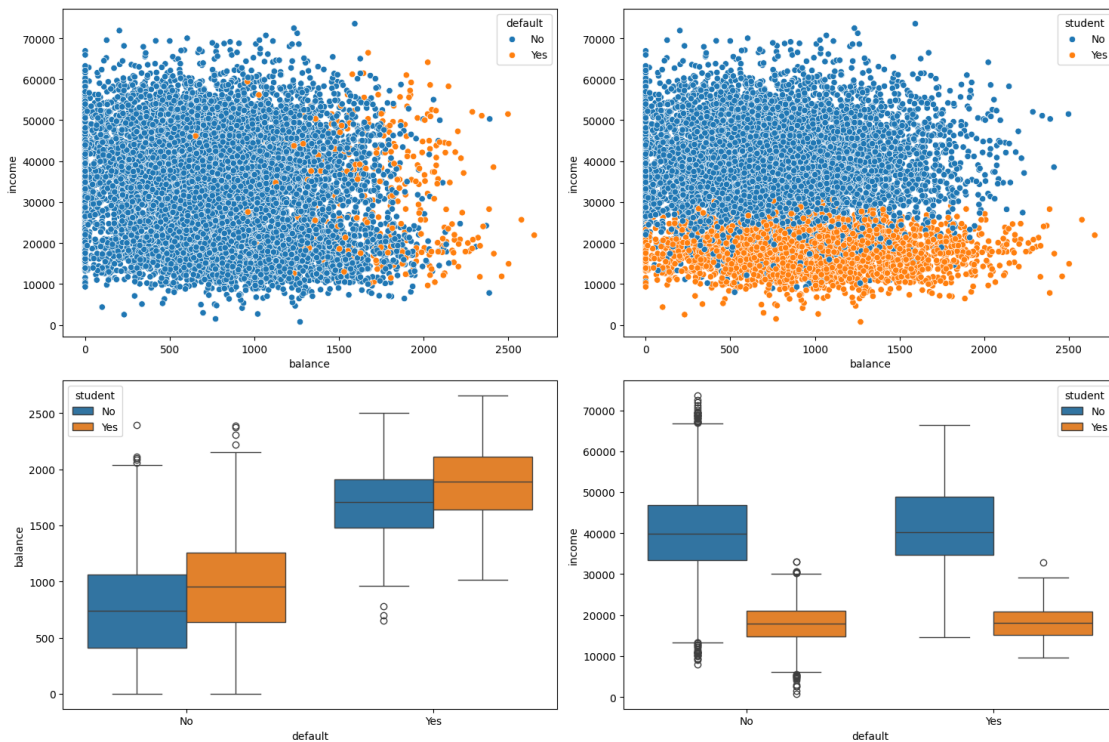
	balance	income
count	10000.000000	10000.000000
mean	835.374886	33516.981876
std	483.714985	13336.639563
min	0.000000	771.967729
25%	481.731105	21340.462905
50%	823.636973	34552.644800
75%	1166.308387	43807.729275
max	2654.322576	73554.233500

Part a. - balance and income are numerical variables, with very different scales. We may need to standardize (especially if using regularization). - default and student are both objects and will need to be encoded. - there are no missing values

```
[6]: # Visualize the data
```

```
fig, axes = plt.subplots(ncols = 2, nrows=2, figsize=(15,10))

sns.scatterplot(data = df_default, x = "balance", y="income", hue="default",
               ↪ax=axes[0,0])
sns.scatterplot(data = df_default, x = "balance", y="income", hue="student",
               ↪ax=axes[0,1])
sns.boxplot(data = df_default, x = "default", y="balance", hue = "student",
            ↪ax=axes[1,0])
sns.boxplot(data = df_default, x = "default", y="income", hue = "student",
            ↪ax=axes[1,1])
plt.tight_layout()
plt.show()
```



Visually, we observe that those who default tend to have a higher credit card balance, and students who default tend to have a higher balance. Students generally have lower income, but there appears to be little relationship between income and default.

2.5 Splitting and Preprocessing the Data

Next, let's create our feature matrix and response vector. We use `LabelEncoder` to encode our categorical output to a binary vector.

```
[7]: from sklearn.preprocessing import LabelEncoder

# Feature matrix and response vector
X, y = df_default.drop(['default'], axis=1), df_default['default']

# Encode default
y = LabelEncoder().fit_transform(y)

print(X.shape)
print(y.shape)

# Print the class distribution before splitting the data set
print(pd.Series(y).value_counts(normalize=True)*100)
```

```
(10000, 3)
(10000,)
0    96.67
1     3.33
Name: proportion, dtype: float64
```

Notice the high class imbalance, with 96.67% of individuals that have not defaulted on their loan. Consider the following naive train/test split.

```
[8]: # Naively split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True,
                                                    test_size = 0.1,
                                                    random_state=1112)

# Check the proportion of defaults in both train and test data sets
print(pd.Series(y_train).value_counts(normalize=True)*100)
print(pd.Series(y_test).value_counts(normalize=True)*100)
```

```
0    96.522222
1     3.477778
Name: proportion, dtype: float64
0    98.0
1     2.0
Name: proportion, dtype: float64
```

2.5.1 Exercise 2 (CORE)

- Why might you NOT want to use the train/test split above?
- Modify the code to have similar class proportions in the train and test sets. Print the class proportions to check if they are similar.

Hint: consider the the additional argument `stratify=` inside of the `train_test_split` function to stratify in terms of the response when splitting the data.

- a. Only 2% of individuals defaulted in the test dataset, while 3.48% defaulted in the training set. In absolute numbers, this difference may seem small, but the relative difference is quite large and can cause poor model performance. Indeed, the test data only contains 20 individuals who defaulted.

```
[9]: print(pd.Series(y_test).value_counts())
```

```
0    980
1     20
Name: count, dtype: int64
```

```
[10]: # Stratify split
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle= True,
↪stratify=y,
                                                    test_size = 0.1,
↪random_state=1112)
```

```
# Check the proportion of defaults in both train and test data sets
print(pd.Series(y_train).value_counts(normalize=True)*100)
print(pd.Series(y_test).value_counts(normalize=True)*100)
```

```
0    96.666667
1     3.333333
Name: proportion, dtype: float64
0    96.7
1     3.3
Name: proportion, dtype: float64
```

3 Logistic Regression

Recall from our notes that for a binary output $y \in \{0, 1\}$, **logistic regression** is a classifier that can be seen a simple generalization of linear regression by making two changes.

- First, we replaced the **Gaussian** distribution of the output y with a **Bernoulli** distribution.
- Second, we pass the linear function of the inputs, $\mathbf{w}^T \mathbf{x}$, through a **link function** $g : \mathbb{R} \rightarrow [0, 1]$.

That is, we assume $y \sim \text{Bern}(g(\mathbf{w}^T \mathbf{x}))$.

The link function takes values in the unit interval to ensure that the conditional probability of a success,

$$p(y = 1 \mid \mathbf{x}) = E[y \mid \mathbf{x}] = g(\mathbf{w}^T \mathbf{x})$$

is between zero and one. Specifically, in logistic regression, we select the **logistic** link function (S-shaped), defined as

$$g(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

Putting these steps together, the logistic regression model is:

$$y \sim \text{Bern} \left([1 + \exp(-\mathbf{w}^T \mathbf{x})]^{-1} \right)$$

In `sklearn`, we can fit a logistic regression model using `LogisticRegression`.

Creating a logistic regression model: some important options include - `penalty` specifies the type of regularization. Options are `l1`, `l2`, `elasticnet`, `None`. - `C` the inverse strength of the penalty parameter. - `'l1_ratio'` the additional penalty parameter when using `elasticnet`.

CAUTION: by default `LogisticRegression` uses an `l2` regularization with penalty parameter `C=1`. This default should never be used! Instead, you should either set `penalty=None` or tune the value of `C` with cross-validation.

Fitting a logistic regression model: - use the usual `.fit(X,y)` to fit the model. The fitted object stores the intercept and coefficients in the usual attributes `.intercept_` and `.coef_`.

Prediction: methods include - `predict` which predicts the class label (either 0 or 1), - `predict_proba` which predicts the class probabilities, and - `predict_log_proba` which predicts the log probabilities of each class.

Let's start by create a pipeline to for our logisitic regression model (with no penatly).

```
[11]: # Pipeline
log_pipe = make_pipeline(
    ColumnTransformer(
        [("cat", OneHotEncoder(drop=["No"]), ['student']),
         ("num", StandardScaler(), ['balance', 'income'])],
        verbose_feature_names_out=False
    ),
    LogisticRegression(random_state=42, penalty=None)
)
```

3.0.1 Exercise 3 (CORE)

- Fit the logistic regression model to the training data
- Run the code to plot the coefficients and comment on their interpretation.
- Compute the the accuracy score of the model on the testing data

Hint

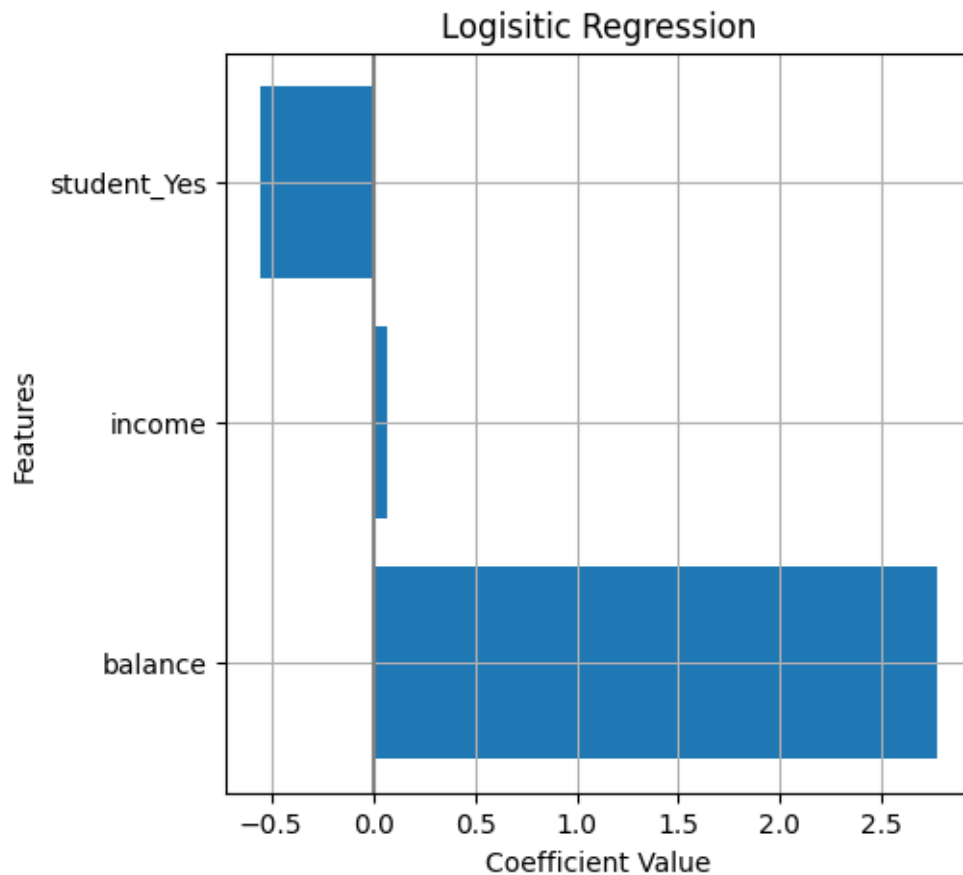
You can use the `score` method of `LogisiticRegression` to compute the accuracy.

```
[12]: # Fit the model using the training data
log_pipe.fit(X_train, y_train)

# accuracy on the test data
print('Accuracy on the test data:', log_pipe.score(X_test,y_test))
```

Accuracy on the test data: 0.975

```
[13]: # Plot logistic regression coefficients
w_df = get_coefs(log_pipe, plot=True, intercept=False, figtitle="Logisitic_Regression")
```



```
[14]: w_df
```

```
[14]:
```

	feature	coef
2	balance	2.773691
3	income	0.070656
1	student_Yes	-0.553895

Interpreting the coefficients: - if the individual is a student, the odds of defaulting on their loan decreases by a factor of $\exp(-0.55)$. - if the credit balance is increased by one standard deviation, the odds of defaulting increases by a factor of $\exp(2.77)$. - the effect of income is negligible.

3.1 Evaluating Classification Models: Beyond Accuracy

Because of the imbalanced nature of the data, looking at accuracy alone is misleading. Indeed, a naive classifier that predicts no one defaults would achieve a high accuracy of 96.7%.

A binary classifier can make two types of errors:

- Incorrectly assigning an individual **who defaults** to the **no default** category (FN)
- Incorrectly assigning an individual who **does not default** to the **default** category (FP)

To better understand the FN vs FP tradeoff, we can compute the confusion matrix. Recall from our notes, the quantities reported in the confusion matrix are:

$$\begin{aligned} \text{TP} &= \sum_{n=1}^N \mathbb{I}(y_n = 1) \mathbb{I}(\hat{y}_n = 1), & \text{FP} &= \sum_{n=1}^N \mathbb{I}(y_n = 0) \mathbb{I}(\hat{y}_n = 1) \\ \text{FN} &= \sum_{n=1}^N \mathbb{I}(y_n = 1) \mathbb{I}(\hat{y}_n = 0), & \text{TN} &= \sum_{n=1}^N \mathbb{I}(y_n = 0) \mathbb{I}(\hat{y}_n = 0) \end{aligned}$$

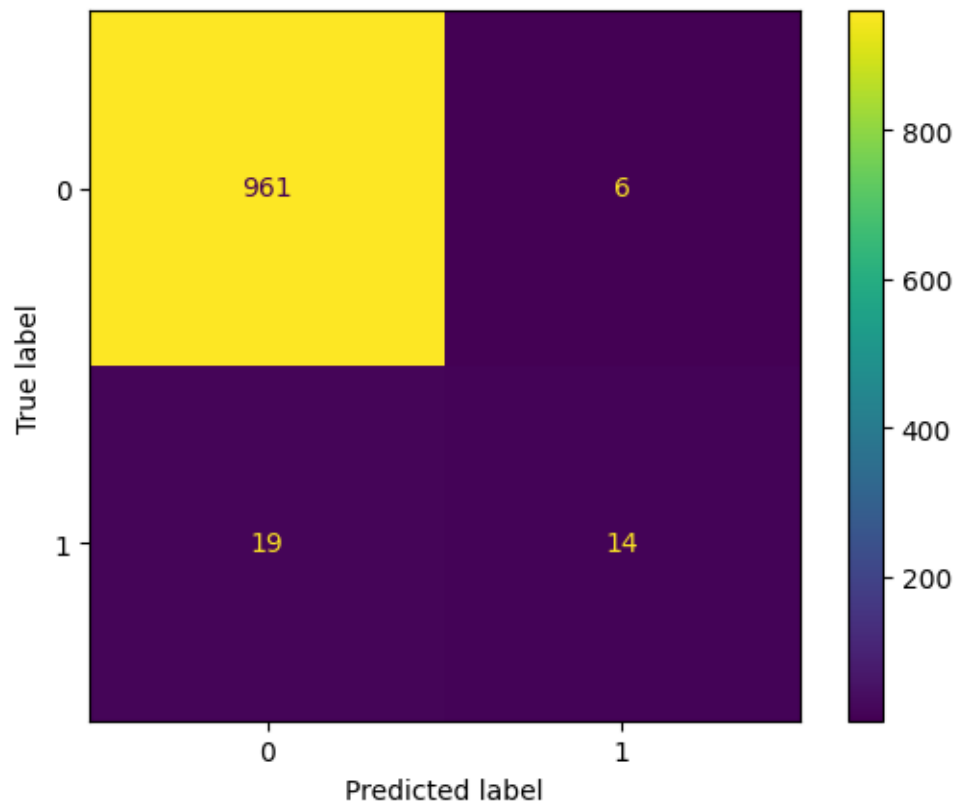
where y_n is the true class and $\hat{y}_n$ is the estimated class.

3.1.1 Exercise 4 (CORE)

1. Use `ConfusionMatrixDisplay.from_estimator` to compute and visualize the confusion matrix.
2. Comment on the importance of FNs compared to FPs to the credit card company.

```
[15]: from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(log_pipe, X_test, y_test)
plt.show()
```



From the perspective of the credit card company, **FN** are more important, as the company may have to take on the entire amount borrowed. Instead, **FP** represents a loss of profit, that is typically much less the financial burden of FN. In this case, FN=19 and FP=6, and the company may prefer higher FP to reduce the number of FN.

Recall from our notes, the different evaluation measures can be defined from the confusion matrix, such as:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad \text{Recall (TPR)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{F1-Score} = 2 \left(\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right) = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

To compute these quantities and more, functions are available in `sklearn.metrics`, including:

1. [Recall](#) (True Positive Rate)
2. [Precision](#)
3. [F1-score](#)
4. [AUC](#)

For a detailed list, see https://scikit-learn.org/1.5/modules/model_evaluation.html#classification-metrics.

3.1.2 Exercise 5 (CORE)

Compute the accuracy, recall, precision, and F1-score on the test data for the fitted model (using the functions in `sklearn.metrics`) and comment on the model's performance.

Hint you first need to use `.predict()` to predict the class labels.

```
[16]: from sklearn.metrics import accuracy_score, precision_score, f1_score, \
      ↪ recall_score

      # Predict the class labels
      y_test_pred = log_pipe.predict(X_test)

      # Compute metrics
      print("Accuracy:", accuracy_score(y_test, y_test_pred))
      print("Recall:", recall_score(y_test, y_test_pred))
      print("Precision:", precision_score(y_test, y_test_pred))
      print("F1-score:", f1_score(y_test, y_test_pred))
```

Accuracy: 0.975
Recall: 0.424242424242425
Precision: 0.7
F1-score: 0.5283018867924528

Again, the model has a high accuracy, but this is misleading given the imbalanced nature of the data. In particular, the recall rate is quite low (and is less than 0.5 as $FN > TP$), which may be unacceptable for the company. Because of this, the F1-score is also low.

3.1.3 Exercise 6 (EXTRA)

Based on the confusion matrix computed below. Use the equations above to derive:

1. False Positive Rate
2. Recall
3. Precision
4. F1-score

without using any additional built-in function from any module. If computed correctly, the numbers should match the previous exercise.

```
[17]: from sklearn.metrics import confusion_matrix

# Predict class labels
y_test_pred = log_pipe.predict(X_test)

# Compute confusion matrix
confmat = confusion_matrix(y_true = y_test, y_pred=y_test_pred)
confmat
```

```
[17]: array([[961,   6],
           [ 19,  14]])
```

```
[18]: # Extracting the quantities from the confusion matrix
FP = confmat[0,1]
TN = confmat[0,0]
TP = confmat[1,1]
FN = confmat[1,0]

#Calculation of FPR, Recall, Precision and F1 Score
# FPR
FPR = (FP)/(FP+TN)
# Recall
Recall = (TP)/(TP+FN)
# Precision
Precision = (TP)/(FP+TP)
# F1 Score
F1 = 2 * (Precision * Recall)/(Precision + Recall)
```

```

print('False Positive Rate (FPR): '+ '%.3f' % FPR)
print('Recall: '+ '%.3f' % Recall)
print('Precision: '+ '%.3f' % Precision)
print('F1 Score: '+ '%.3f' % F1)

```

```

False Positive Rate (FPR): 0.006
Recall: 0.424
Precision: 0.700
F1 Score: 0.528

```

3.1.4 Exercise 7 (CORE)

The handy function `RocCurveDisplay.from_estimator` is also available in `sklearn` for plotting ROC curve for the fitted model. Use this function to plot the ROC curve and compute the AUC using `sklearn.metrics.roc_auc_score` on the test data. Comment on the model's performance based on these quantities.

Hint you will need to use `predict_proba()` to compute the class probabilities for each data point, in order to compute the AUC.

```

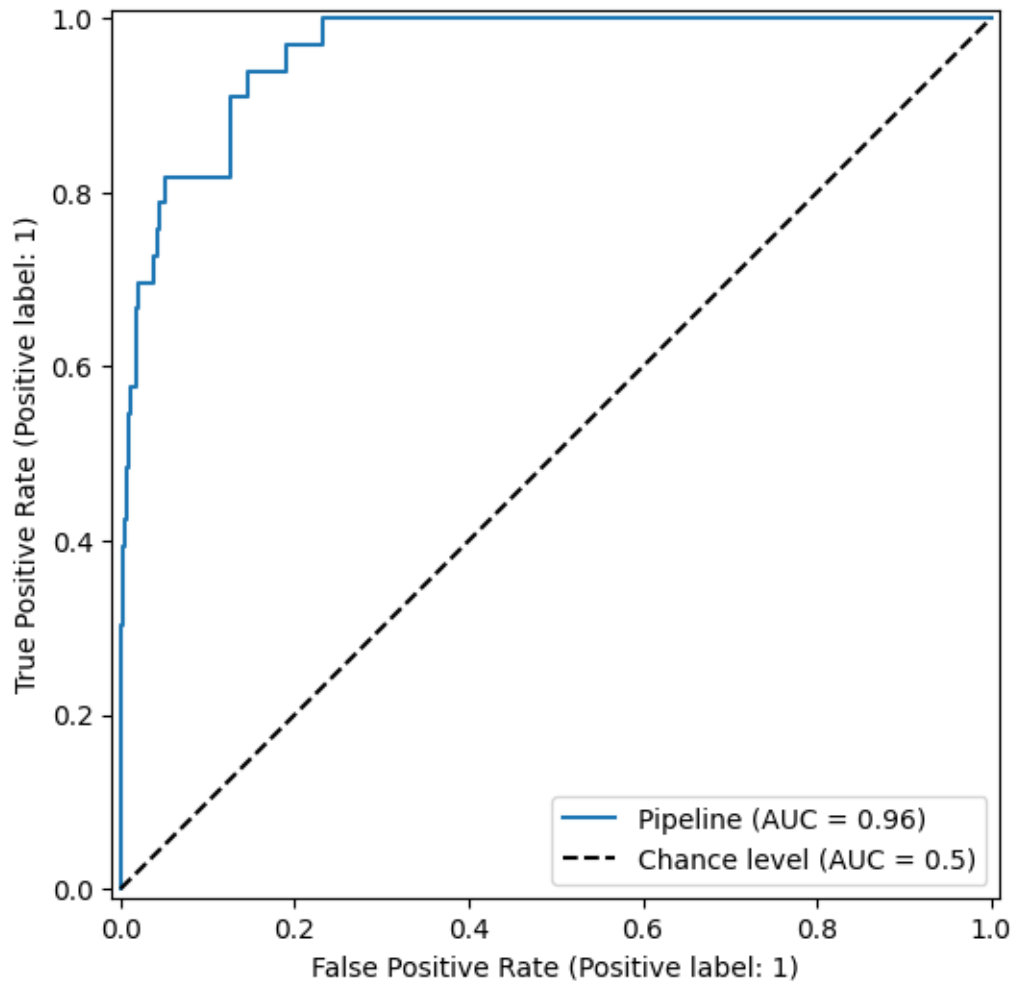
[19]: from sklearn.metrics import RocCurveDisplay, roc_auc_score

# Plot the ROC curve
fig, ax = plt.subplots(figsize=(6,6))
RocCurveDisplay.from_estimator(log_pipe, X_test, y_test, plot_chance_level=True,
    ↪ax = ax)
plt.show()

# Compute the probabilities for each class on the test data (N_test by C matrix)
y_test_prob = log_pipe.predict_proba(X_test)

# Compute the AUC
print('AUC:', roc_auc_score(y_test, y_test_prob[:,1]))

```



AUC: 0.9618000062674313

Based on the ROC curve and AUC, the performance of the model appears very good. The ROC curve hugs the top left corner and the AUC is close to 1. However, as we know from lectures, the ROC and AUC can be misleading on imbalanced data, to the large number of negatives.

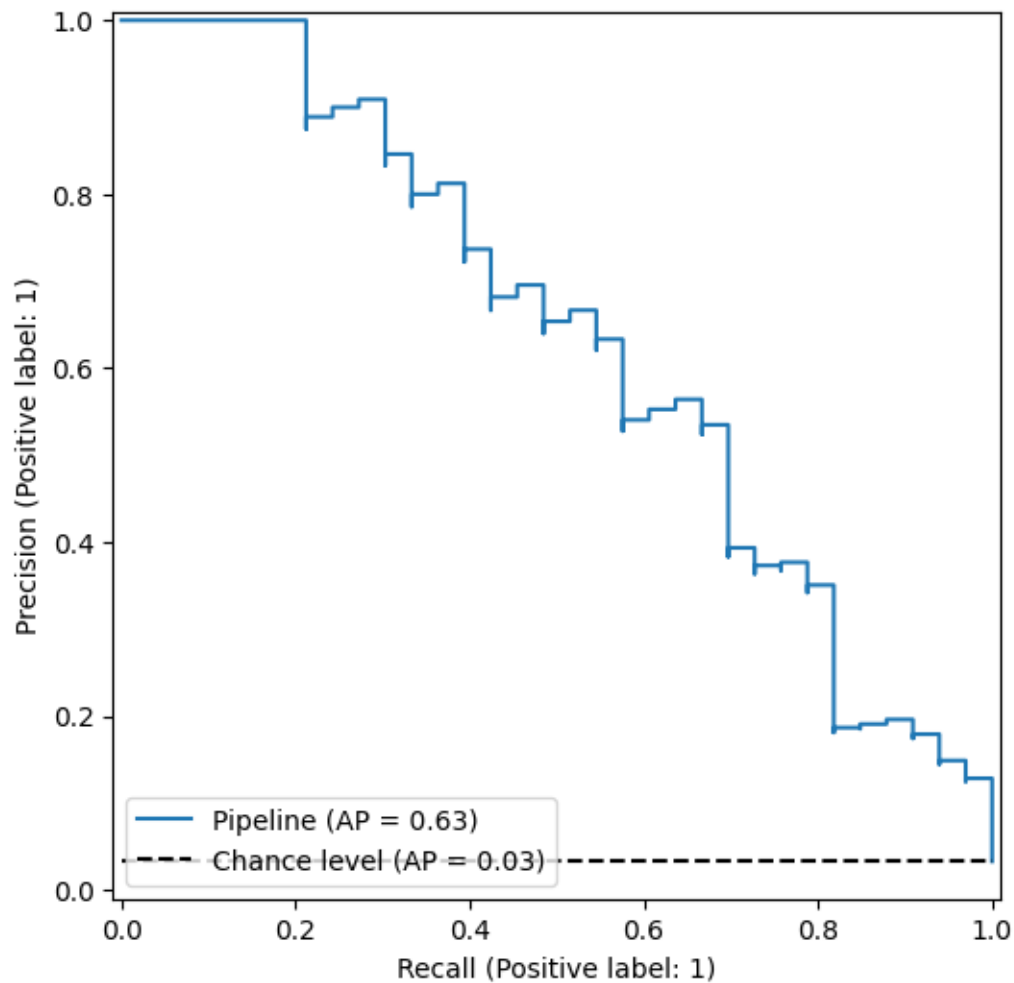
3.1.5 Exercise 8 (CORE)

Now, use the function `PrecisionRecallDisplay.from_estimator` to plot the Precision-Recall curve for the fitted model on the test data. Comment on the model's performance based on this figure.

```
[20]: from sklearn.metrics import PrecisionRecallDisplay

fig, ax = plt.subplots(figsize=(6,6))
PrecisionRecallDisplay.from_estimator(log_pipe, X_test,
    ↪ y_test, plot_chance_level=True, ax = ax)
```

```
plt.show()
```



The curve is far the ideal setting, i.e. it does not hug the top right corner, but does perform better than chance. In contrast to the ROC curve, the precision-recall curve is more appropriate for this data set due to the class imbalance. It highlights that the model is not very precise when it identifies an individual that defaults.

4 Regularization

Regularization can be useful in logistic regression to deal with high-dimensional data and for variable selection. As previous stated `LogisticRegression` offers four options through the ‘penalty’ parameter: - `penalty=None` corresponds to no regularization, - `penalty=l2` is the default and corresponds to l2 regularization (ridge), - `penalty=l1` corresponds to l1 regularization (lasso), and - `penalty=elasticnet` corresponds to a combination of l1 and l2 regularization.

For the three regularization methods, the parameter `C` represents the inverse strength of the penalty parameter. Additionally, when using `elasticnet`, `l1_ratio` is an additional penalty parameter,

controlling the balance between l1 and l2 regularization.

Note: the choice of the algorithm (aka solver) depends on the penalty chosen (see the [documentation](#) for more details.)

As we saw last week, it is important to tune the penalty parameter C (and `l1_ratio` if using elastic net) when including regularization. Before we start searching over hyperparameters, its worth noting that some of the folds may not have the same distribution of the classes. This is particularly important for imbalanced data and means the we could get a validation score that may be a poor estimate of performance (for example we may have a fold with very few positive classes or more than usual, which can cause large differences on imbalanced data). To address this, when doing our grid search, we will use a `StratifiedKFold` to ensure the distribution of classes in our folds reflects the distribution in the larger data.

Run the code below to investigate the difference in class distributions across folds when using `Kfold` vs `StratifiedKfold`.

```
[21]: # Consider 5 folds
KF = KFold(n_splits=5)
SKF = StratifiedKFold(n_splits=5)

fold_names = ["KFold", "StratifiedKFold"]
for i, K in enumerate([KF, SKF]):
    # Initialize an empty DataFrame to store counts for the current fold type
    fold_nos = pd.DataFrame()
    for j, (train_i, test_i) in enumerate(K.split(X_train, y_train)):
        # Compute value counts for the current fold and ensure it's a DataFrame
        #with appropriate columns
        fold_no = pd.DataFrame(pd.Series(y_train[test_i]).value_counts()).T
        fold_no.index = ["Fold " + str(j)] # Rename the index to reflect the
        #fold number
        # Concatenate with the fold_nos DataFrame
        fold_nos = pd.concat([fold_nos, fold_no], axis=0)

    fold_nos.fillna(0, inplace=True) # Fill missing values with 0 if any class
    #was not present in a fold
    print(f"{fold_names[i]} counts per fold:\n", fold_nos)
```

KFold counts per fold:

	0	1
Fold 0	1744	56
Fold 1	1730	70
Fold 2	1746	54
Fold 3	1742	58
Fold 4	1738	62

StratifiedKFold counts per fold:

	0	1
Fold 0	1740	60
Fold 1	1740	60

```
Fold 2  1740  60
Fold 3  1740  60
Fold 4  1740  60
```

Next, let's perform a grid search to tune the regularization parameter with l2 regularization. Notice that we have listed multiple metrics to save with the option `scoring=`. However, setting `refit=` will select the best model and refit according to the specified metric.

```
[22]: # This code snippet sets up and executes a grid search for tuning the penalty_
      ↪parameter
      # of a logistic regression model with l2 regularization using cross-validation.

      # Pipeline
      log_pipe_l2 = make_pipeline(
          ColumnTransformer(
              [("cat", OneHotEncoder(drop=["No"]), ['student']),
               ("num", StandardScaler(), ['balance', 'income'])],
              verbose_feature_names_out=False
          ),
          LogisticRegression(random_state=42, penalty='l2')
      )

      # Uncomment this line to find how the penalty parameter is called in the_
      ↪pipeline
      #log_pipe.get_params()

      # Possible C values:
      C_list = np.linspace(0.01, 10, num=151)

      # Grid search CV:
      log_rs = GridSearchCV(log_pipe_l2,
                           param_grid={'logisticregression__C': C_list},
                           scoring = ["accuracy", "f1", "recall", "precision"],_
                           ↪#Evaluation metrics to compute on validation sets
                           cv = StratifiedKFold(n_splits=5, shuffle=True),
                           refit = "accuracy", # Refits the best model on the entire_
                           ↪dataset using the accuracy metric
                           return_train_score = True)

      # Tune the model with grid search:
      log_rs.fit(X_train, y_train)
```

```
/home/darrenljs/MachineLearninginPython_UoE/.venv/lib/python3.12/site-
packages/sklearn/metrics/_classification.py:1706: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

```
[22]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=None, shuffle=True),
                  estimator=Pipeline(steps=[('columntransformer',
                                             ColumnTransformer(transformers=[('cat',
OneHotEncoder(drop=['No']),
['student']),
('num',
StandardScaler(),
['balance',
'income'])]),
verbose_feature_names_out=False)),
                  ('logisticregression',
                  LogisticRegression(random_state=42))]),
        param_g...
        7.4692,  7.5358,  7.6024,  7.669 ,  7.7356,  7.8022,  7.8688,
        7.9354,  8.002 ,  8.0686,  8.1352,  8.2018,  8.2684,  8.335 ,
        8.4016,  8.4682,  8.5348,  8.6014,  8.668 ,  8.7346,  8.8012,
        8.8678,  8.9344,  9.001 ,  9.0676,  9.1342,  9.2008,  9.2674,
        9.334 ,  9.4006,  9.4672,  9.5338,  9.6004,  9.667 ,  9.7336,
        9.8002,  9.8668,  9.9334, 10.    ]}),
        refit='accuracy', return_train_score=True,
        scoring=['accuracy', 'f1', 'recall', 'precision'])
```

4.0.1 Exercise 9 (CORE)

- Run the following code to plot the mean accuracy averaged across the validation folds (in black). Comment on the suggested value of C.
- Choose a metric other than accuracy (i.e. f1, recall, or precision) and redraw the figure to plot the mean across the validation folds. Does your suggested value of C change?

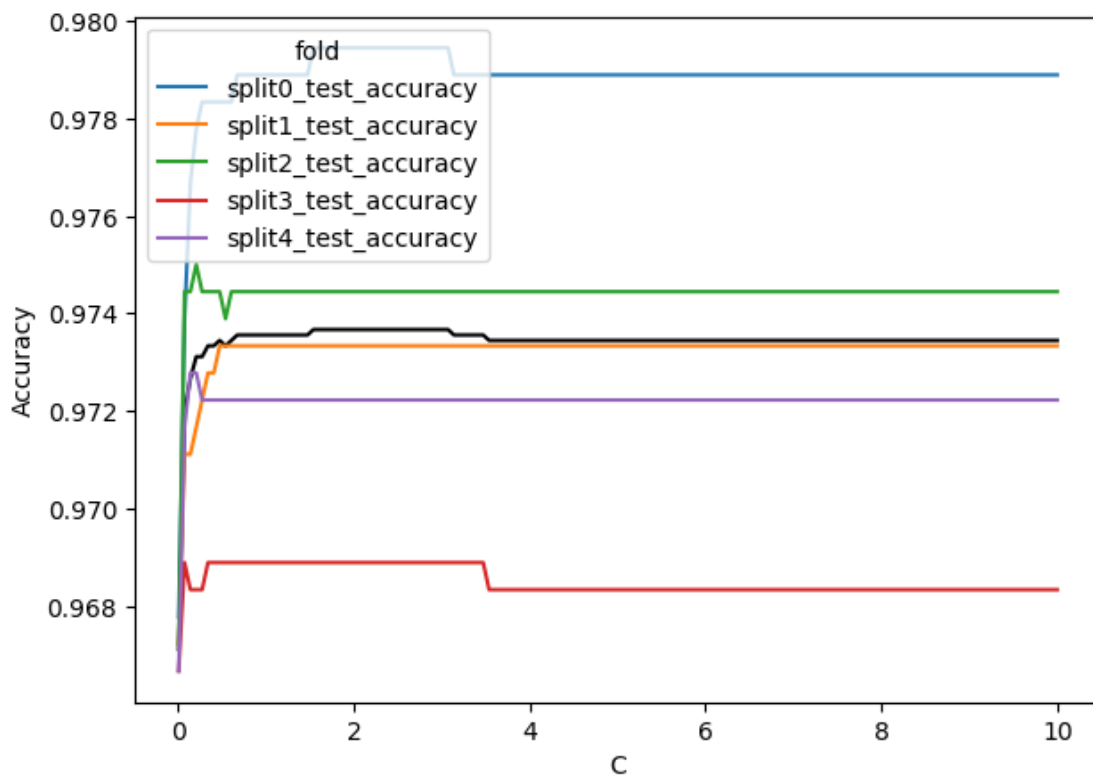
```
[23]: # Extract only mean and split scores
cv_accuracy = pd.DataFrame(
    data = log_rs.cv_results_
).filter(
    # Extract the split#_test_accuracy and mean_test_accuracy columns
    regex = '(split[0-4]+|mean)_test_accuracy'
).assign(
    # Add the alphas as a column
    C = C_list
)

# Reshape the data frame for plotting
d = cv_accuracy.melt(
    id_vars=('C', 'mean_test_accuracy'),
    var_name='fold',
    value_name='Accuracy'
)
```

```

# Plot the validation scores across folds
plt.figure(figsize=(7,5))
sns.lineplot(x='C', y='Accuracy', color='black', errorbar=None, data = d) # Plot the mean score in black.
sns.lineplot(x='C', y='Accuracy', hue='fold', data = d) # Plot the curves for each fold in different colors
plt.show()

```



Looking at accuracy, the mean accuracy is fairly constant across C values, and any value except the smallest would be reasonable. We also notice that regularization provides little improvement in terms of accuracy and little variability across folds (the scale of the y-axis is quite small).

```

[24]: # Repeat for f1

# Extract only mean and split scores
cv_accuracy = pd.DataFrame(
    data = log_rs.cv_results_
).filter(
    # Extract the split#_test_f1 and mean_testf1 columns
    regex = '(split[0-4]+|mean)_test_f1'
).assign(
    # Add the alphas as a column

```

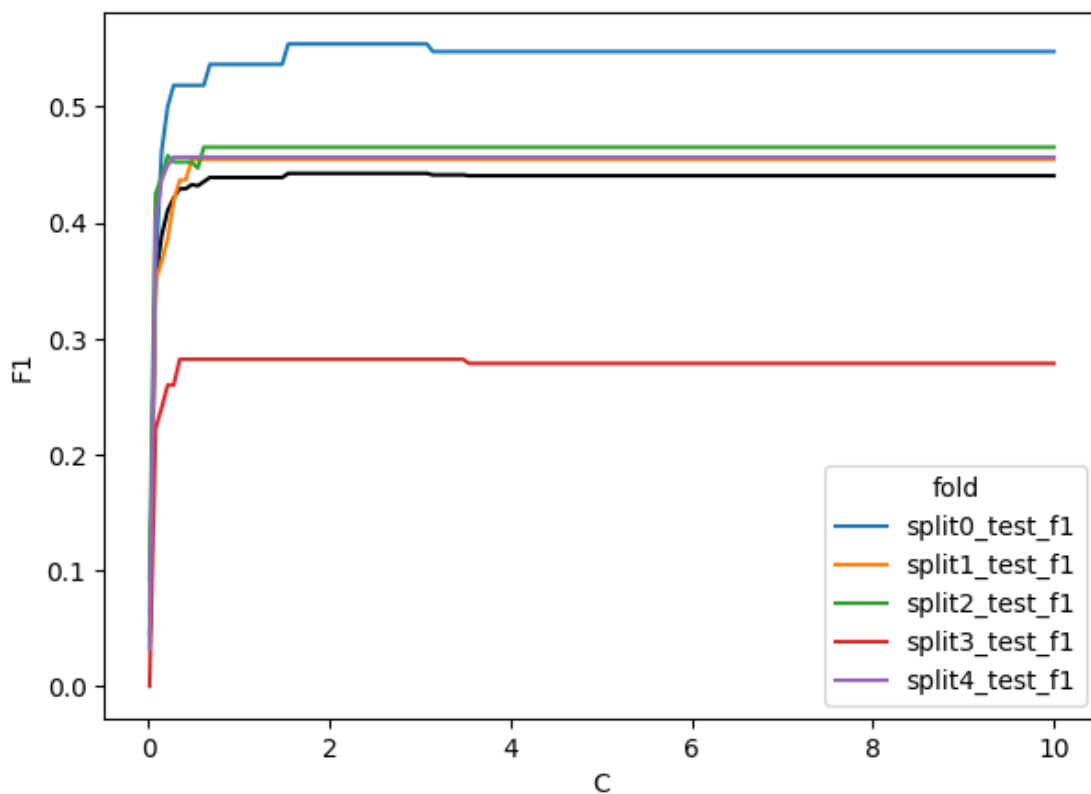
```

    C = C_list
)

# Reshape the data frame for plotting
d = cv_accuracy.melt(
    id_vars=('C', 'mean_test_f1'),
    var_name='fold',
    value_name='F1'
)

# Plot the validation scores across folds
plt.figure(figsize=(7,5))
sns.lineplot(x='C', y='F1', color='black', errorbar=None, data = d) # Plot the
    ↪ mean score in black.
sns.lineplot(x='C', y='F1', hue='fold', data = d) # Plot the curves for each
    ↪ fold in different colors
plt.show()

```



When looking at other metrics, such as the F1 score (which is more appropriate for imbalanced data), our conclusions do not change. Regularization seems to have little effect, and any value of C larger than 1 would be reasonable. We do observe more variability across folds.

4.0.2 Exercise 10 (EXTRA)

- Rebuild your model with l1 regularization and perform a grid search to tune the penalty parameter. Which metric have you chosen for refitting the model and why?
- Plot the coefficients using the helper function. Which variables are included? How does the performance compare to the model with no regularization?

```
[25]: # Pipeline
log_pipe_l1 = make_pipeline(
    ColumnTransformer(
        [("cat", OneHotEncoder(drop=["No"]), ['student']),
         ("num", StandardScaler(), ['balance', 'income'])],
        verbose_feature_names_out=False
    ),
    LogisticRegression(random_state=42, penalty='l2')
)

# Possible C values:
C_list = np.linspace(0.01, 3, num=101)

# Grid search CV:
log_rs_l1 = GridSearchCV(log_pipe_l1,
                          param_grid={'logisticregression__C': C_list},
                          scoring = ["accuracy", "f1", "recall", "precision"],
                          ↪#Evaluation metrics to compute on validation sets
                          cv = StratifiedKFold(n_splits=5, shuffle=True),
                          refit = "f1", # Refits the best model on the entire
                          ↪dataset using the f1 metric
                          return_train_score = True)

# Tune the model with grid search:
log_rs_l1.fit(X_train, y_train)
```

```
[25]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=None, shuffle=True),
                  estimator=Pipeline(steps=[('columntransformer',
                                             ColumnTransformer(transformers=[('cat',
OneHotEncoder(drop=['No']),
                                             ['student']),
                                             ('num',
StandardScaler(),
                                             ['balance',
'income'])],
verbose_feature_names_out=False)),
                                             ('logisticregression',
                                             LogisticRegression(random_state=42))])),
                  param_g...
```

```

1.6844, 1.7143, 1.7442, 1.7741, 1.804 , 1.8339, 1.8638, 1.8937,
1.9236, 1.9535, 1.9834, 2.0133, 2.0432, 2.0731, 2.103 , 2.1329,
2.1628, 2.1927, 2.2226, 2.2525, 2.2824, 2.3123, 2.3422, 2.3721,
2.402 , 2.4319, 2.4618, 2.4917, 2.5216, 2.5515, 2.5814, 2.6113,
2.6412, 2.6711, 2.701 , 2.7309, 2.7608, 2.7907, 2.8206, 2.8505,
2.8804, 2.9103, 2.9402, 2.9701, 3.    ]}},
    refit='f1', return_train_score=True,
    scoring=['accuracy', 'f1', 'recall', 'precision'])

```

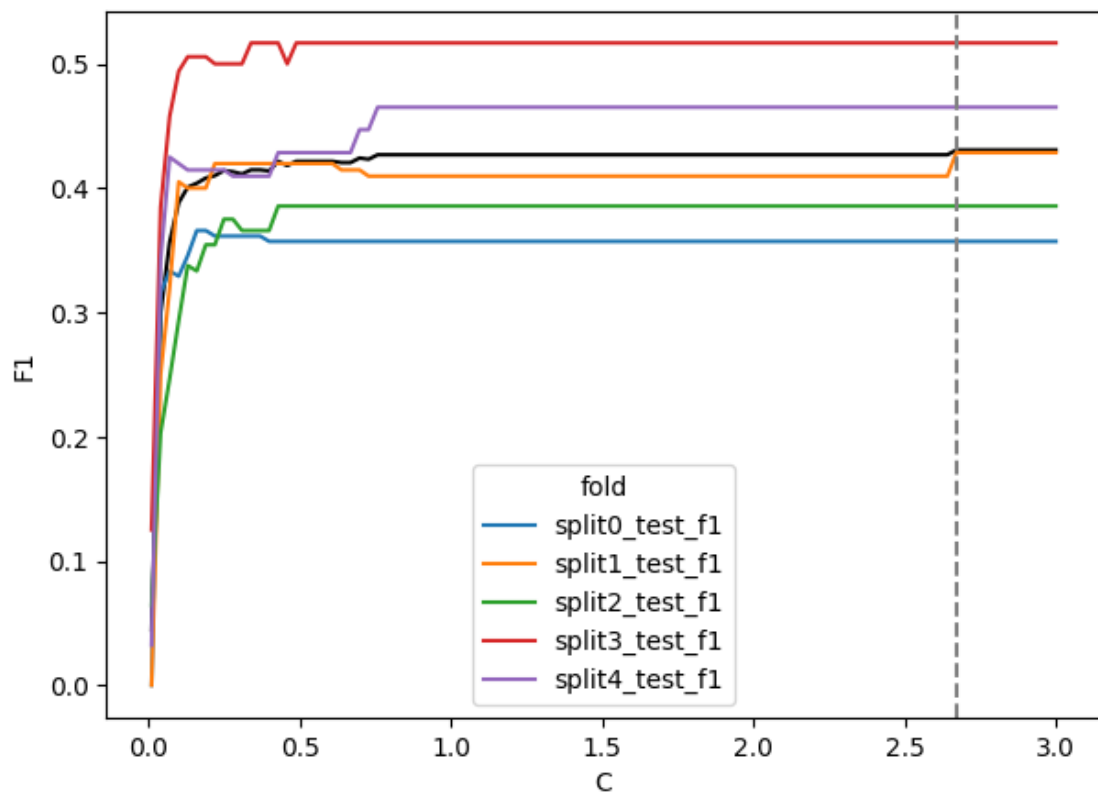
```

[26]: # Extract only mean and split scores
cv_accuracy = pd.DataFrame(
    data = log_rs_l1.cv_results_
).filter(
    # Extract the split#_test_f1 and mean_testf1 columns
    regex = '(split[0-4]+|mean)_test_f1'
).assign(
    # Add the alphas as a column
    C = C_list
)

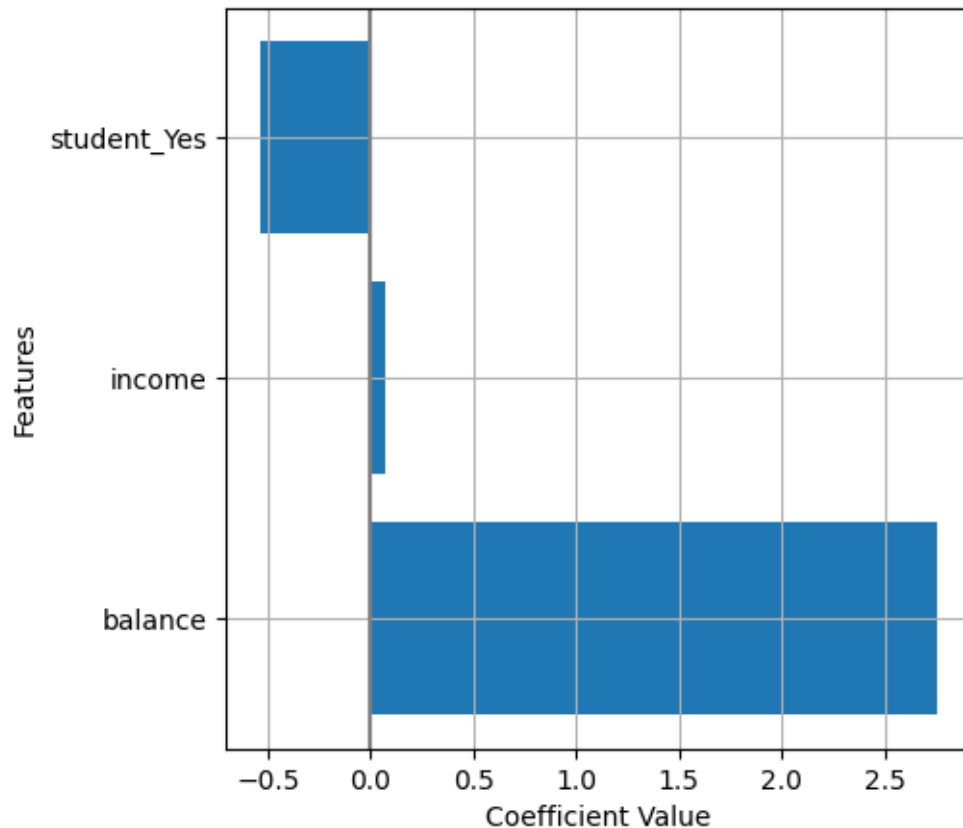
# Reshape the data frame for plotting
d = cv_accuracy.melt(
    id_vars=('C', 'mean_test_f1'),
    var_name='fold',
    value_name='F1'
)

# Plot the validation scores across folds
plt.figure(figsize=(7,5))
sns.lineplot(x='C', y='F1', color='black', errorbar=None, data = d) # Plot the
    ↪ mean score in black.
sns.lineplot(x='C', y='F1', hue='fold', data = d) # Plot the curves for each
    ↪ fold in different colors
plt.axvline(log_rs_l1.best_params_['logisticregression_C'], 0,1, linestyle =
    ↪ '--', c = "grey")
plt.show()

```



```
[27]: get_coefs(log_rs_l1.best_estimator_, plot=True, intercept = False)
```



```
[27]:      feature      coef
      2      balance  2.757629
      3      income  0.074746
      1  student_Yes -0.536316
```

```
[28]: # Compute metrics on the test data
y_test_pred = log_rs_l1.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Recall:", recall_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred))
print("F1-score:", f1_score(y_test, y_test_pred))
```

```
Accuracy: 0.975
Recall: 0.42424242424242425
Precision: 0.7
F1-score: 0.5283018867924528
```

Due to the imbalanced nature of the data, we have chosen the F1 metric over accuracy to refit the model.

All variables are included in the model, and the performance on the test data is the same as when the model was fit with no regularization. Again, we find that regularization offers no improvement.

5 Dealing with Imbalanced Data

We have already seen that when dealing with imbalanced data, we need to 1. Considering a suitable performance metric both for testing and validation. 2. Use a suitable splitting strategy (stratified splitting) both in creating our test and validation sets.

But the majority class can still overwhelm the minority class when fitting the model. To investigate, let's use the function `classification_report`, which reports summary of the precision, recall, F1 score for each class. Note that: - the recall of the positive class is also known as *sensitivity*; recall of the negative class is *specificity*. - it also includes the *macro average* (averaging the unweighted mean per label) as well as the weighted average (averaging the support-weighted mean per label).

```
[29]: # Compute and print the classification report for the model with no penalty
from sklearn.metrics import classification_report

print(classification_report(y_test,
                           log_pipe.predict(X_test),
                           target_names = ['No Default', 'Default']))
```

	precision	recall	f1-score	support
No Default	0.98	0.99	0.99	967
Default	0.70	0.42	0.53	33
accuracy			0.97	1000
macro avg	0.84	0.71	0.76	1000
weighted avg	0.97	0.97	0.97	1000

Notice, how the metrics are much higher for the majority class (individuals who have not defaulted). In this case, we may want to alter the training algorithm or data by:

1. Weighting the classes during training.
2. Resampling the data.

`LogisticRegression` offers the option to include class weights, and for example `class_weight="balanced"` will weight the classes by: $N/(2 N_c)$, where N_c for $c = 0, 1$ counts the number of observations in each class. Notice how the metrics in classification report change when including this option in the code below.

```
[30]: log_pipe_balan = make_pipeline(
    ColumnTransformer(
        [
            ("cat", OneHotEncoder(drop=["No"]), ['student']),
            ("num", StandardScaler(), ['balance', 'income'])
        ],
        verbose_feature_names_out=False
    ),
    LogisticRegression(random_state=42, penalty=None, class_weight='balanced')
).fit(X_train, y_train)

print(classification_report(y_test,
```

```
log_pipe_balan.predict(X_test),
target_names = ['No Default', 'Default']))
```

	precision	recall	f1-score	support
No Default	1.00	0.87	0.93	967
Default	0.19	0.91	0.32	33
accuracy			0.87	1000
macro avg	0.59	0.89	0.62	1000
weighted avg	0.97	0.87	0.91	1000

In the following, the focus is on suitable **resampling** to change the distribution of the classes in our training data.

5.1 Resampling

To alter the distribution of the classes in our training data, there are two main approaches:

- Under-sampling the majority class
- Over-sampling the minority class

We will be using Imbalanced-learn (imported as `imblearn`), an open source, MIT-licensed library relying on scikit-learn that provides tools for dealing with classification with imbalanced classes (see [here](#) for an introduction). Let's start by installing the `imblearn` package if necessary.

```
[31]: # Install the imblearn if necessary
      #!pip install imblearn
```

In the previous notebooks, we learned about *transformers*, which allow us to **alter the features** and not the number of observations in our data (i.e. the columns of **X**). Instead, *resamplers* provide a preprocessing step in our Pipeline to **alter the number of observations** and not the features (i.e. the rows of **X** and **y**).

Resamplers

Resamplers are classes that follow the scikit-learn API and have a sampling functionality through the `.resample()` method. Like all other scikit-learn methods, they have a `.fit()` method which is only applied during pipeline training. This means if we want to create our own resampler from scratch that is compatible with scikit-learn, we just have to make a class that has three methods; `.fit()`, `.resample()`, and `.fit_resample()`; with the latter just chaining the other two together.

Therefore to resample a dataset, each sampler implements:

```
obj.fit(data, targets)
data_resampled, targets_resampled = obj.resample(data, targets)
```

or simply...

```
data_resampled, targets_resampled = obj.fit_resample(data, targets)
```

Remember to include your sampler within your model pipeline to prevent data leakage!

5.1.1 Under-Sampling

Under-sampling involves removing observations from the majority class to prevent its signal from dominating during training. We will focus on the simplest strategy:

- `RandomUnderSampler`, which randomly removes observations from the majority class.

For an overview of other under-samplers, see: https://imbalanced-learn.org/stable/under_sampling.html

```
[32]: from imblearn.pipeline import Pipeline as ImPipeline
      from imblearn.under_sampling import RandomUnderSampler

      # Pipeline with under sampling
      log_pipe_us = ImPipeline([
          ("fe", ColumnTransformer(
              [("cat", OneHotEncoder(drop=["No"]), ['student']),
               ("num", StandardScaler(), ['balance', 'income'])],
              verbose_feature_names_out=False
          )),
          ("sampler", RandomUnderSampler(random_state=42)),
          ("model", LogisticRegression(random_state=42, penalty=None))]

      # Fit model
      log_pipe_us.fit(X_train, y_train)
```

```
[32]: Pipeline(steps=[('fe',
                        ColumnTransformer(transformers=[('cat',
                                                         OneHotEncoder(drop=['No']),
                                                         ['student']),
                                                         ('num', StandardScaler(),
                                                         ['balance', 'income'])],
                                                         verbose_feature_names_out=False)),
                        ('sampler', RandomUnderSampler(random_state=42)),
                        ('model', LogisticRegression(penalty=None, random_state=42))])
```

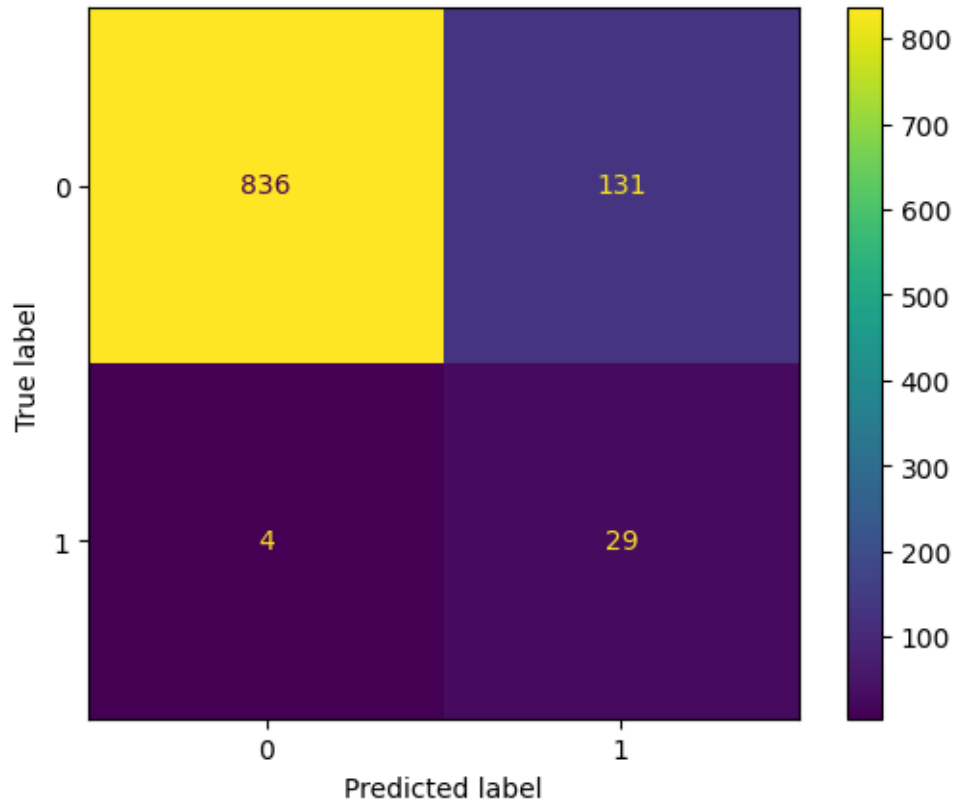
5.1.2 Exercise 11 (CORE)

Compute and print the classification report and visualize the confusion matrix for the model above with undersampling. How have the results changed?

```
[33]: # Print classification report
      print(classification_report(y_test,
                                  log_pipe_us.predict(X_test),
                                  target_names = ['No Default', 'Default']))

      # Draw confusion matrix
      ConfusionMatrixDisplay.from_estimator(log_pipe_us, X_test, y_test)
      plt.show()
```

	precision	recall	f1-score	support
No Default	1.00	0.86	0.93	967
Default	0.18	0.88	0.30	33
accuracy			0.86	1000
macro avg	0.59	0.87	0.61	1000
weighted avg	0.97	0.86	0.90	1000



We notice that among the default cases, the precision and f1-score are worse but the recall is much higher. In particular, the model is better able to identify individuals that default but this comes at the cost of higher FP (not loaning to individuals who will not default on their loans).

5.2 Oversampling

Over-sampling involves generating additional observations from the minority class. We will focus on the simplest strategy:

- [RandomOverSampler](#), which randomly samples with replacement observations from the minority class.

For an overview of other over-samplers, see: <https://imbalanced->

learn.org/stable/over_sampling.html

5.2.1 Exercise 12 (CORE)

- Create a new pipeline for a logistic regression model with oversampling (by copying and editing the pipeline above).
- How do the performance metrics compare with under-sampling?

```
[34]: from imblearn.over_sampling import RandomOverSampler

# Pipeline with over sampling
log_pipe_os = ImPipeline([
    ("fe", ColumnTransformer(
        [("cat", OneHotEncoder(drop=["No"]), ['student']),
        ("num", StandardScaler(), ['balance', 'income'])],
        verbose_feature_names_out=False
    )),
    ("sampler", RandomOverSampler(random_state=42)),
    ("model", LogisticRegression(random_state=42, penalty=None))])

# Fit the model
log_pipe_os.fit(X_train, y_train)
```

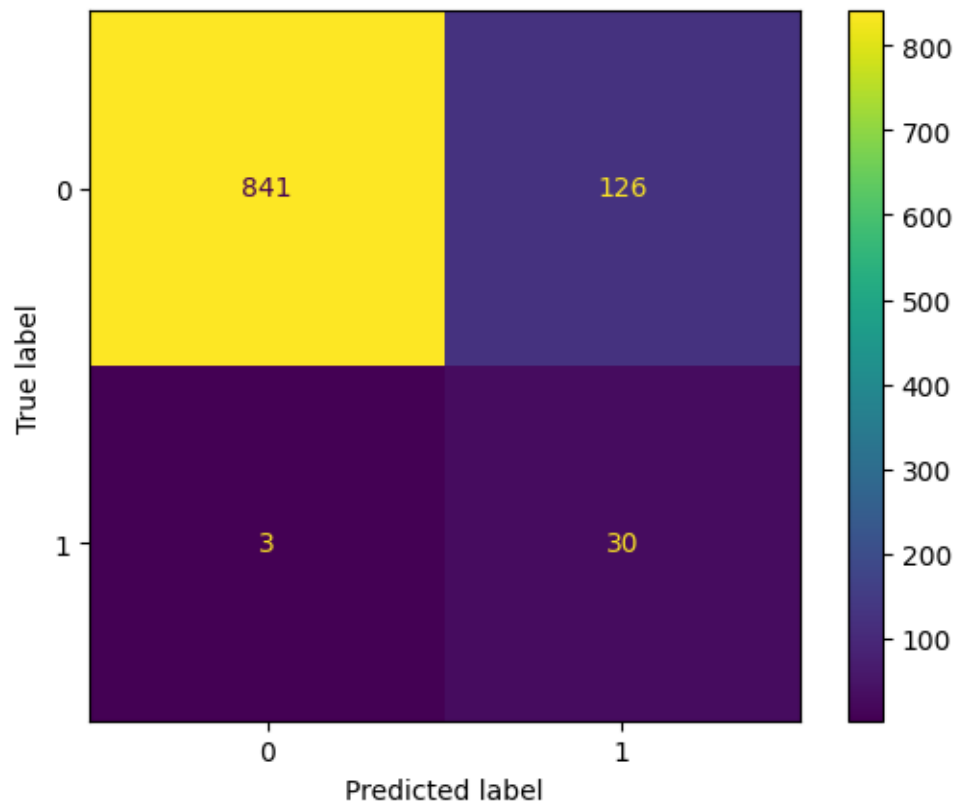
```
[34]: Pipeline(steps=[('fe',
                        ColumnTransformer(transformers=[('cat',
                                                         OneHotEncoder(drop=['No']),
                                                         ['student']),
                                                         ('num', StandardScaler(),
                                                         ['balance', 'income'])],
                                                         verbose_feature_names_out=False)),
                        ('sampler', RandomOverSampler(random_state=42)),
                        ('model', LogisticRegression(penalty=None, random_state=42))])
```

```
[35]: # Print the classification report
print(classification_report(y_test,
                            log_pipe_os.predict(X_test),
                            target_names = ['No Default', 'Default']))

# Visualize the confusion matrix
ConfusionMatrixDisplay.from_estimator(log_pipe_os, X_test, y_test)
plt.show()
```

	precision	recall	f1-score	support
No Default	1.00	0.87	0.93	967
Default	0.19	0.91	0.32	33
accuracy			0.87	1000

macro avg	0.59	0.89	0.62	1000
weighted avg	0.97	0.87	0.91	1000



The results are slightly better with over-sampling. The number of TP is increased from 29 to 30, and both the numbers of FP and FN are slightly lower.

6 Multi-class Logistic Regression

Finally, to gain some experience with multi-class logistic regression, let's now look at the `iris` data set, which we have seen already in lectures.

This is an example where the response has **3 classes** corresponding to the three types of iris species and $D = 4$ features (petal length, petal width, sepal length, sepal width).

Let's start by loading the data and doing some basic EDA.

```
[36]: # First load the data
      from sklearn.datasets import load_iris

      # Loading data
```

```
iris = pd.DataFrame(sns.load_dataset('iris'))

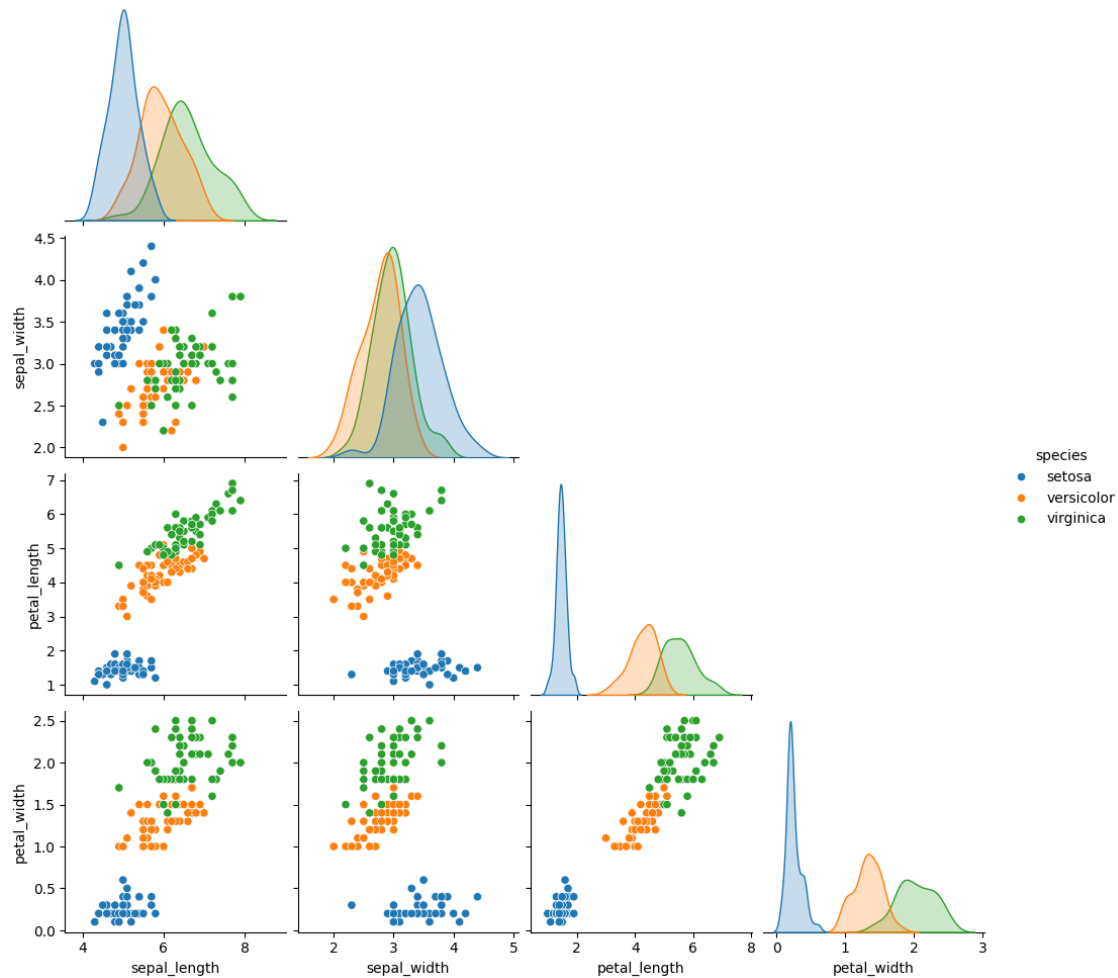
# Print information about the data set
print(iris.info())
print(iris.describe())
print(iris['species'].value_counts())

# Pairplot
sns.pairplot(data = iris, hue = 'species', corner=True)
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
species
setosa      50
versicolor  50
virginica   50
Name: count, dtype: int64
```



From the initial EDA, we observe: - The features have similar, but slightly different scales. - Visually, the species appear to be fairly well separated. - The target variable **species** is a string and we will need a label encoding - The species types are equal distributed, and thus no methods for class imbalance are needed.

Now, let's separate the features and target, encode the target, and split into training and test sets.

```
[37]: # Feature matrix and response vector
X, y = iris.drop(['species'], axis=1), iris['species']

# Encode target
le = LabelEncoder()
y = le.fit_transform(y)

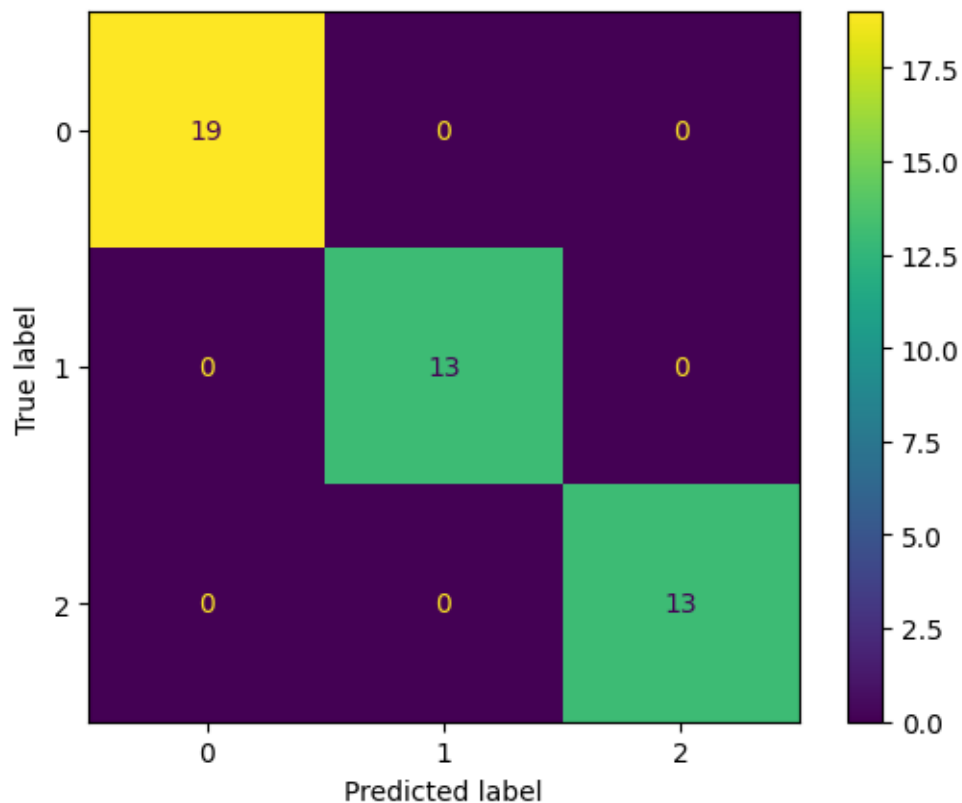
# Split into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42, shuffle=True)
```

6.0.1 Exercise 13 (CORE)

We can use the same `LogisticRegression` for the multi-class setting. In this case, it will fit the **multinomial logistic regression** model. Fit a multinomial logistic regression model and visualize the confusion matrix.

```
[38]: # Multinomial logistic regression
lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(penalty=None, random_state=42))
lr.fit(X_train, y_train)

# Plot the confusion matrix
ConfusionMatrixDisplay.from_estimator(lr, X_test, y_test)
plt.show()
```



6.0.2 Exercise 14 (EXTRA)

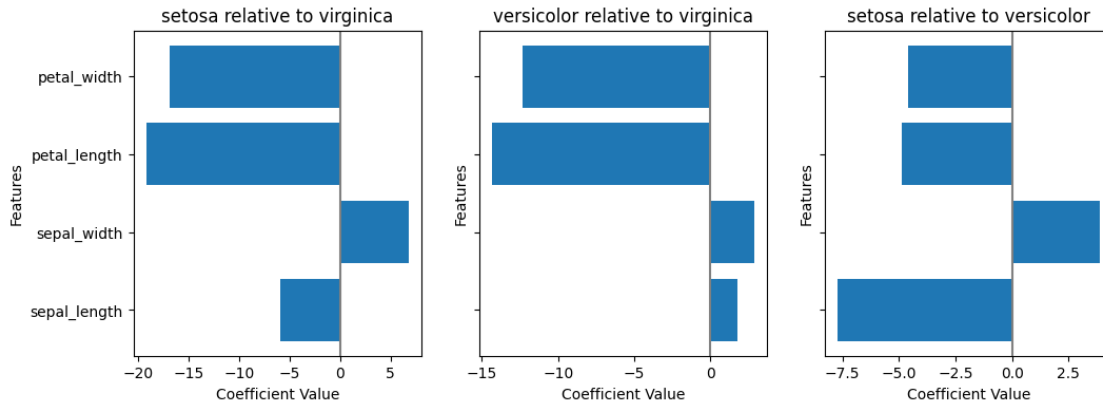
In the multi-class setting, the `.coeff_` attribute is a $C \times D$ matrix of the model coefficients. Recall that the log odds of being in class c relative to class c' is:

$$\log \left(\frac{p(y = c | \mathbf{x})}{p(y = c' | \mathbf{x})} \right) = \log \left(\frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{l=1}^C \exp(\mathbf{w}_l^T \mathbf{x})} / \frac{\exp(\mathbf{w}_{c'}^T \mathbf{x})}{\sum_{l=1}^C \exp(\mathbf{w}_l^T \mathbf{x})} \right) = (\mathbf{w}_c - \mathbf{w}_{c'})^T \mathbf{x}$$

For each unique pair of classes, create a barplot to visualize the coefficients of one class relative to another. Which features are most important for distinguishing between classes? Interpret how a unit change in the features affects the odds of being in one class relative to another.

```
[39]: # Extract coefficients relative to a reference category
fe_names = lr[:-1].get_feature_names_out()
w0_ref2 = lr[-1].coef_[0,:]- lr[-1].coef_[2,:]
w1_ref2 = lr[-1].coef_[1,:]- lr[-1].coef_[2,:]
w0_ref1 = lr[-1].coef_[0,:]- lr[-1].coef_[1,:]

# Plot coefficients
fig, ax = plt.subplots(1,3,figsize=(12,4),sharey=True)
w02_df = pd.DataFrame({'feature': fe_names, 'coef': w0_ref2})
ax[0].barh(w02_df['feature'], w02_df['coef'])
ax[0].set_ylabel('Features')
ax[0].set_xlabel('Coefficient Value')
ax[0].axvline(x=0, color=".5")
ax[0].set_title(f'{le.classes_[0]} relative to {le.classes_[2]}')
w12_df = pd.DataFrame({'feature': fe_names, 'coef': w1_ref2})
ax[1].barh(w12_df['feature'], w12_df['coef'])
ax[1].set_ylabel('Features')
ax[1].set_xlabel('Coefficient Value')
ax[1].axvline(x=0, color=".5")
ax[1].set_title(f'{le.classes_[1]} relative to {le.classes_[2]}')
w01_df = pd.DataFrame({'feature': fe_names, 'coef': w0_ref1})
ax[2].barh(w01_df['feature'], w01_df['coef'])
ax[2].set_ylabel('Features')
ax[2].set_xlabel('Coefficient Value')
ax[2].axvline(x=0, color=".5")
ax[2].set_title(f'{le.classes_[0]} relative to {le.classes_[1]}')
plt.show()
```



There are three unique pairs of classes.

- Setosa relative to virginica: the most important features are petal length and petal width. A unit increase in these features decreases the odds of flower being a setosa relative to virginica by a factor of approximately $\exp(-19)$ and $\exp(-16)$, respectively.
- Versicolor relative to virginica: the most important features are again petal length and petal width. A unit increase in these features decreases the odds of flower being a versicolor relative to virginica by a factor of approximately $\exp(-14)$ and $\exp(-12)$, respectively.
- Setosa relative to versicolor: the most important features is sepal length. A unit increase in this features decreases the odds of flower being a setosa relative to versicolor by a factor of approximately $\exp(-8)$.

7 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

```
[40]: # !jupyter nbconvert --to pdf mlp_week06_key.ipynb
```